



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Framework with FastAPI

Eng. Fatma Elraey

Lab 2 — Harden the API with Validation, Uploads & Errors

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lab 2 of 5
Focus	Refactor the Documents API to use Pydantic request/response models, file uploads, environment variables, and structured errors.
Evaluation	Lab Assignment — 100%

Before You Start

This lab continues the same Documents API used across the course. Keep each step working before moving on, and test every endpoint as you build it.

Today's Lab Plan

1. Install validation/upload dependencies
2. Create request and response schemas
3. Observe automatic 422 validation
4. Add file upload endpoint
5. Load .env configuration
6. Standardize application errors

Install

```
pip install pydantic python-multipart python-dotenv
```

01 Create Pydantic Schemas

Goal: Replace unstructured dict request bodies with explicit contracts.

Steps

1. Create schemas.py.
2. Define DocumentCreate with title, content and priority.
3. Define DocumentResponse with id plus the public fields.
4. Import the models into main.py.

Starter / reference code

```
# schemas.py
from pydantic import BaseModel, Field

class DocumentCreate(BaseModel):
    title: str = Field(min_length=3, max_length=120)
    content: str = Field(min_length=1)
    priority: int = Field(default=1, ge=1, le=5)

class DocumentResponse(BaseModel):
    id: int
    title: str
    content: str
    priority: int
```

✓ CHECKPOINT — Exercise 1

Q1. Which field is server-owned?

Answer: *id; it belongs in the response but not the create request.*

02 Validate POST & PUT

Goal: Use typed Pydantic bodies and response_model.

Steps

1. Change POST to accept payload: DocumentCreate.
2. Return response_model=DocumentResponse.
3. Use payload.model_dump() when constructing the stored record.
4. Send invalid title/content/priority values and inspect 422.

Starter / reference code

```
@app.post(
    "/documents",
    response_model=DocumentResponse,
    status_code=201,
)
def create_document(payload: DocumentCreate):
    new_id = max(documents.keys(), default=0) + 1
    item = {"id": new_id, **payload.model_dump()}
    documents[new_id] = item
    return item
```

✓ CHECKPOINT — Exercise 2

Q1. Does invalid input enter the endpoint body?

Answer: *No. FastAPI/Pydantic rejects it first.*

03 Add a File Upload Endpoint

Goal: Accept text/plain or application/pdf uploads and return metadata.

Steps

1. Install python-multipart if not already installed.
2. Add POST /documents/upload.
3. Reject unsupported content types with HTTP 400.
4. Return filename, content_type and byte size.

Starter / reference code

```
from fastapi import File, UploadFile, HTTPException

@app.post("/documents/upload")
async def upload_document(file: UploadFile = File(...)):
    if file.content_type not in {"text/plain", "application/pdf"}:
        raise HTTPException(status_code=400, detail="Unsupported file type")
    content = await file.read()
    return {
        "filename": file.filename,
        "content_type": file.content_type,
        "size": len(content),
    }
```

✓ CHECKPOINT — Exercise 3

Q1. Which content type does a file form use?

Answer: *multipart/form-data.*

04 Move Configuration to .env

Goal: Load application settings from environment variables.

Steps

1. Create .env with APP_NAME and MAX_UPLOAD_MB.
2. Call load_dotenv() during startup/config import.
3. Read values with os.getenv().
4. Create .env.example and ensure secret-bearing .env is ignored by Git.

Starter / reference code

```
# .env
APP_NAME=Documents API
MAX_UPLOAD_MB=5

# main.py or settings.py
import os
from dotenv import load_dotenv

load_dotenv()
APP_NAME = os.getenv("APP_NAME", "Documents API")
MAX_UPLOAD_MB = int(os.getenv("MAX_UPLOAD_MB", "5"))
```

✓ CHECKPOINT — Exercise 4

Q1. What belongs in .env.example?

Answer: *Variable names and safe placeholders, not real secrets.*

05

Structured Application Errors

Goal: Add a custom AppError and exception handler.

Steps

1. Create AppError with code, message and status_code.
2. Register @app.exception_handler(AppError).
3. Use the error for one explicit business rule, such as duplicate title or disallowed operation.
4. Verify the JSON error structure in Swagger/curl.

Starter / reference code

```
from fastapi import Request
from fastapi.responses import JSONResponse

class AppError(Exception):
    def __init__(self, code: str, message: str, status_code: int = 400):
        self.code = code
        self.message = message
        self.status_code = status_code

@app.exception_handler(AppError)
async def handle_app_error(request: Request, exc: AppError):
    return JSONResponse(
        status_code=exc.status_code,
        content={"error": {"code": exc.code, "message": exc.message}},
    )
```

✓ CHECKPOINT — Exercise 5

Q1. Why is a structured error better than arbitrary strings?

Answer: Clients can reliably inspect known fields.

06

Submission Checklist

- DocumentCreate and DocumentResponse exist.
- POST and PUT use typed bodies.
- Invalid payloads produce 422.
- File upload endpoint accepts allowed types and rejects others.
- .env configuration loads successfully.
- At least one custom structured AppError is returned and tested.